

ShiftingBrowserFingerprints: A Python Package for Browser Fingerprint Obfuscation

Peter A. Fockema ^{1,2*}

¹ Infoserv Systems, United Kingdom ² BCS (British Computer Society), United Kingdom * These authors contributed equally.

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: 

Submitted: 15 August 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Browser fingerprinting (also known as device or online fingerprinting) is a family of techniques that is utilised on the majority of the top 1000 websites (in terms of traffic used online). The fingerprinting software used by these types of websites has increased in resilience to methods of obfuscation of their calculated results of visitor ids (the final calculation of a fingerprint id).

The fingerprinting introduced by these websites are often used to target users for advertising or to identify repeat users visiting websites which can be flagged as web scrapers.

The ability to 'dupe' browser fingerprinting therefore has value to users that want to repeatedly pull page data from specific websites without maintaining a singular fingerprint that can be used to track their activity across the website and, if the fingerprints are shared with 3rd parties, across multiple websites. This can reduce the user's susceptibility to unwanted marketing or profiling by employers of fingerprint tracking.

The field of fingerprinting, which aims to identify users on a website and assign them a visitor id to track their activities or interests is now well-established and, as mentioned above, commonly used, so a software tool that can help obfuscate the replicability of the visitor id calculated when a visitor accesses a website using browser fingerprinting is of increasing interest.

Statement of need

We can first consider how browser fingerprinting is applied, where the website the user visits will run a script when the site is accessed, typically as JavaScript in the browser ([Fingerprint, 2026b](#); [Fingerprint, 2026a](#)). The purpose here is to collect and store particular information about the computer being used ([Hibbert, 2022](#)).

The information collected can include the Operating System (e.g. whether the visitor uses a Windows machine, or a Linux variety), the browser in question (e.g. Firefox, Chrome, Edge, or IceWeasel), the language in use (e.g. English, French), screen resolution, font size, etc. Various techniques have been created to collect and use these types of data to calculate a reasonably accurate unique identifier for a site visitor.

Whilst some individual fingerprinting techniques can provide more unique identifiers than others (e.g. canvas fingerprinting is touted as having, depending on the source, between 80% and 99% accuracy ([Ganz, 2024](#))), if they are combined with other fingerprinting techniques then there can be a 99.99+% accuracy. ([Omisola, 2025](#)) The result is that when fingerprinting uses a collection of techniques it can become highly accurate (where only 1 in 286777 browsers share the same fingerprint with other users on the internet ([Eckersley, 2010](#))).

Browser fingerprinting usage has increased at a significant rate in the last decade, where

39 fingerprinting occurred on less than 1% of the 10000 most visited websites in 2013, yet by
40 2021 a quarter were employing browser fingerprinting techniques. (["Your web browsing habits
41 may be less private than you think," 2023](#))

42 FingerprintObfuscation was designed for stress-testing fingerprinting software against chosen
43 obfuscation techniques and was developed as a fingerprinting- interference Python package
44 that can call these techniques to interfere with active browser fingerprinting against a user
45 that is implementing browser scraping via a set Python script.

46 State of the field

47 Several tools exist for browser fingerprint obfuscation, typically in the form of browser extensions/
48 add-ons for Chrome or Firefox. These include, but are not limited to:

- 49 ▪ AudioContext Scrambler ([Sayrus, 2020](#))
- 50 ▪ CanvasBlocker ([kkapsner, 2026](#))
- 51 ▪ Font Fingerprint Defender ([ilGur, 2026a](#))
- 52 ▪ WebGL Fingerprint Defender ([ilGur, 2026b](#))
- 53 ▪ Fingerprint Defender ([Fracture, 2026](#))

54 Tools like these maintain many of the capabilities we want to implement, however they are
55 only implemented as browser add-ons and require implementation via the browser (so lack
56 the integration into a Python package or the ability to add or remove based on dynamic user
57 preferences).

58 With regards to python packages that have similar goals, there are the

59 The primary existing tools that 'dupe' fingerprinting techniques exist primarily as browser
60 add-ons supplied through platforms such as Mozilla and Chrome.

61 FingerprintObfuscation was built rather than contributing to existing projects for multiple
62 reasons. First, FingerprintObfuscation was designed from the ground up to integrate in
63 a web scraping architecture, by calling and instantiating web drivers on behalf of the user.
64 Through FingerprintObfuscation we can supply different configurations of the web drivers
65 for the user to run against a defined web URL endpoint to scrape data from whilst with
66 fingerprint obfuscation techniques (of the user's choice) switched on.

67 This integration between fingerprint obfuscation through use of a Python package fills a specific
68 niche between simple demonstration and full codes for web-scraping software and fingerprint
69 obfuscation through browser add-ons that would usually be added by the user, often through
70 the web interface. This allows the user to maintain performance through automation and
71 usability through the FingerprintObfuscation interface.

72 Software design

73 FingerprintObfuscation is a Python package which is designed with the following core
74 principles in mind: (1) Provide a user-friendly, modular, object-oriented API (2) Use community
75 tools and standards (PEP8 coding standards) (3) Maximum control for the user to call
76 obfuscation techniques of choice The package supplies the obfuscation methods for the
77 following fingerprinting techniques: * Audio * Battery * Canvas * ClientRects * Font *
78 Navigator * Screen * WebGL * WebRTC

79 By combining these techniques we can observe the effects as they are layered and applied to
80 the user's browser as Addons. This package currently only supports the Firefox browser. These
81 Addons can be called individually for the purposes of web scraping or used in conjunction with
82 each other.

(4) Package instantiation on command The user will build the Addon/ Extension files that the package dynamically calls to install on the Firefox browser driver. The extensions are unsigned by design for ease of creation by the end user and as a double lock so they will not persist on the browser after the end of a browser session.

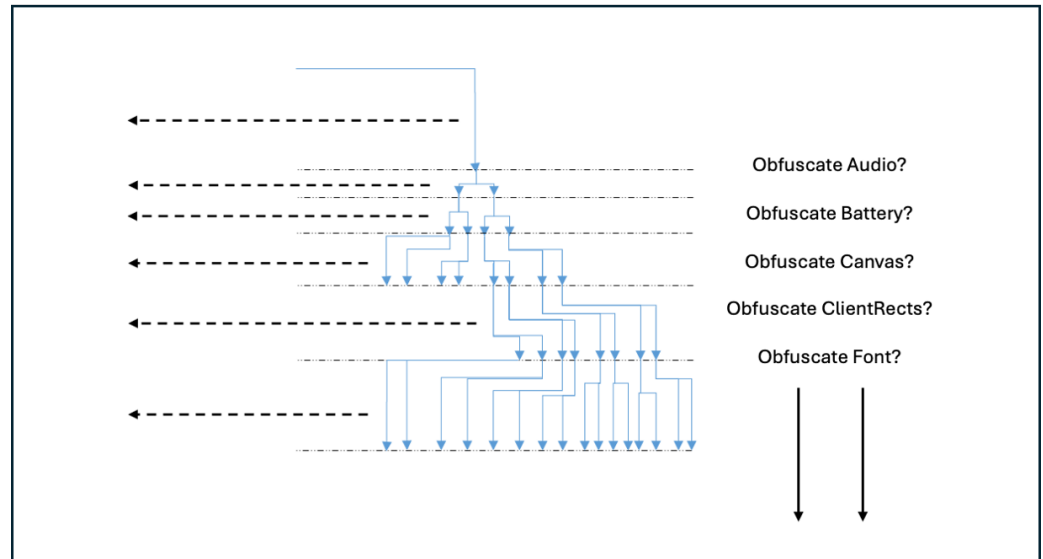


Figure 1: A Visual Representation of how the FingerprintObfuscation Library Should be Deployed Layering Multiple Techniques for Obfuscation.

FingerprintObfuscation should be called by a program implemented using Python for scraping a set website and is designed to not impact the purpose of the underlying program.

The API for FingerprintObfuscation is designed to provide a class-based and user- friendly interface to fast implementations of various browser fingerprinting obfuscation technique implementations in a dynamic manner.

Research impact statement

FingerprintObfuscation has demonstrated a research impact on how to counter known fingerprinting techniques employed by major fingerprinting vendors, like FingerprintJS (Fingerprint, 2026b; Fingerprint, 2026a).

FingerprintObfuscation started as a tool to support the experimentation of how combining multiple techniques together to dupe a combination of at least the techniques defined in the Software Design section.

We saw that successively combining these techniques produced increased variance in the final fingerprint id/ visitor id.

The result was that combining these techniques could produce complete variation in the fingerprint visitor id produced by software like FingerprintJS (Fingerprint, 2026b; Fingerprint, 2026a).that is a widespread fingerprinting vendor that combines a number of fingerprinting techniques to generate a fingerprint, represented by it's visitor_id variable.

We have run the software using the defined testing framework at [browser_fingerprinting_test_framework:2

If we consider the following representation of the results:

Where grey represents the initial fingerprint for the run (which a remained consistent value

throughout all three runs), red represents a run with no variation away from the original fingerprint, and green represents a successful variation away the original fingerprint, we have the following:

[illegible]

[illegible]

114 Notable behaviour can be observed in relation to the canvas fingerprinting interference results.
115 We can observe canvas interference results in the following table.

Tested Combinations of Fingerprints Using Canvas Variation			
No interference		canvas, navigator, screen, webgl	Consistent Variation #4
canvas	Consistent Variation #1	canvas, navigator, screen, webRTC	Consistent Variation #4
audio, canvas		canvas, navigator, webgl, webRTC	Consistent Variation #2
battery, canvas	Consistent Variation #1	canvas, screen, webgl, webRTC	Consistent Variation #3
canvas, clientRects	Consistent Variation #1	audio, battery, canvas, clientRects, font	
canvas, font	Consistent Variation #1	audio, battery, canvas, clientRects, navigator	
canvas, navigator	Consistent Variation #2	audio, battery, canvas, clientRects, screen	
canvas, screen	Consistent Variation #3	audio, battery, canvas, clientRects, webgl	
canvas, webgl	Consistent Variation #1	audio, battery, canvas, clientRects, webRTC	
canvas, webRTC	Consistent Variation #1	audio, battery, canvas, font, navigator	
audio, battery, canvas		audio, battery, canvas, font, screen	
battery, canvas, clientRects	Consistent Variation #1	audio, battery, canvas, font, webgl	
battery, canvas, font	Consistent Variation #1	audio, battery, canvas, font, webRTC	
battery, canvas, navigator	Consistent Variation #2	audio, battery, canvas, navigator, screen	
battery, canvas, screen	Consistent Variation #3	audio, battery, canvas, navigator, webgl	
battery, canvas, webgl	Consistent Variation #1	audio, battery, canvas, navigator, webRTC	
battery, canvas, webRTC	Consistent Variation #1	audio, battery, canvas, screen, webgl	
canvas, clientRects, font	Consistent Variation #1	audio, battery, canvas, screen, webRTC	
canvas, clientRects, navigator	Consistent Variation #2	audio, battery, canvas, webgl, webRTC	
canvas, clientRects, screen	Consistent Variation #3	audio, canvas, clientRects, font, navigator	
canvas, clientRects, webgl	Consistent Variation #1	audio, canvas, clientRects, font, screen	
canvas, clientRects, webRTC	Consistent Variation #1	audio, canvas, clientRects, font, webgl	
audio, battery, canvas, clientRects		audio, canvas, clientRects, font, webRTC	
audio, battery, canvas, font		audio, canvas, clientRects, navigator, screen	
audio, battery, canvas, navigator		audio, canvas, clientRects, navigator, webgl	
audio, battery, canvas, screen		audio, canvas, clientRects, navigator, webRTC	
audio, battery, canvas, webgl		audio, canvas, clientRects, screen, webgl	
audio, battery, canvas, webRTC		audio, canvas, clientRects, screen, webRTC	
audio, canvas, clientRects, font		audio, canvas, clientRects, webgl, webRTC	
audio, canvas, clientRects, navigator		audio, canvas, font, navigator, webgl	
audio, canvas, clientRects, screen		audio, canvas, font, navigator, webRTC	
audio, canvas, clientRects, webgl		audio, canvas, font, screen, webgl	
audio, canvas, clientRects, webRTC		audio, canvas, font, screen, webRTC	
audio, canvas, font, navigator		audio, canvas, font, webgl, webRTC	
audio, canvas, font, screen		audio, canvas, navigator, screen, webgl	
audio, canvas, font, webgl		audio, canvas, navigator, screen, webRTC	
audio, canvas, font, webRTC		audio, canvas, navigator, webgl, webRTC	
audio, canvas, navigator, screen		audio, canvas, screen, webgl, webRTC	
audio, canvas, navigator, webgl		battery, canvas, clientRects, font, navigator	Consistent Variation #2
audio, canvas, navigator, webRTC		battery, canvas, clientRects, font, screen	Consistent Variation #3
audio, canvas, screen, webgl		battery, canvas, clientRects, font, webgl	Consistent Variation #1
audio, canvas, screen, webRTC		battery, canvas, clientRects, font, webRTC	Consistent Variation #1
battery, canvas, clientRects, font	Consistent Variation #1	battery, canvas, clientRects, navigator, screen	Consistent Variation #4
battery, canvas, clientRects, navigator	Consistent Variation #2	battery, canvas, clientRects, navigator, webgl	Consistent Variation #2
battery, canvas, clientRects, screen	Consistent Variation #3	battery, canvas, clientRects, navigator, webRTC	Consistent Variation #2
battery, canvas, clientRects, webgl	Consistent Variation #1	battery, canvas, clientRects, screen, webgl	Consistent Variation #3
battery, canvas, clientRects, webRTC	Consistent Variation #1	battery, canvas, clientRects, screen, webRTC	Consistent Variation #3
battery, canvas, font, navigator	Consistent Variation #2	battery, canvas, clientRects, webgl, webRTC	Consistent Variation #1
battery, canvas, font, screen	Consistent Variation #3	battery, canvas, font, navigator, screen	Consistent Variation #4
battery, canvas, font, webgl	Consistent Variation #1	battery, canvas, font, navigator, webgl	Consistent Variation #2
battery, canvas, font, webRTC	Consistent Variation #1	battery, canvas, font, navigator, webRTC	Consistent Variation #2
battery, canvas, navigator, screen	Consistent Variation #4	battery, canvas, font, screen, webgl	Consistent Variation #3
battery, canvas, navigator, webgl	Consistent Variation #2	battery, canvas, font, screen, webRTC	Consistent Variation #3
battery, canvas, navigator, webRTC	Consistent Variation #2	battery, canvas, font, webgl, webRTC	Consistent Variation #1
battery, canvas, screen, webgl	Consistent Variation #3	battery, canvas, navigator, screen, webgl	Consistent Variation #4
battery, canvas, screen, webRTC	Consistent Variation #3	battery, canvas, navigator, screen, webRTC	Consistent Variation #4
battery, canvas, webgl, webRTC	Consistent Variation #1	battery, canvas, navigator, webgl, webRTC	Consistent Variation #2
canvas, clientRects, font, navigator	Consistent Variation #2	battery, canvas, screen, webgl, webRTC	Consistent Variation #3
canvas, clientRects, font, screen	Consistent Variation #3	canvas, clientRects, font, navigator, screen	Consistent Variation #4
canvas, clientRects, font, webgl	Consistent Variation #1	canvas, clientRects, font, navigator, webgl	Consistent Variation #2
canvas, clientRects, font, webRTC	Consistent Variation #1	canvas, clientRects, font, navigator, webRTC	Consistent Variation #2
canvas, clientRects, navigator, screen	Consistent Variation #4	canvas, clientRects, font, screen, webgl	Consistent Variation #3
canvas, clientRects, navigator, webgl	Consistent Variation #2	canvas, clientRects, font, screen, webRTC	Consistent Variation #3
canvas, clientRects, navigator, webRTC	Consistent Variation #2	canvas, clientRects, font, webgl, webRTC	Consistent Variation #1
canvas, clientRects, screen, webgl	Consistent Variation #3	canvas, clientRects, navigator, screen, webgl	Consistent Variation #4
canvas, clientRects, screen, webRTC	Consistent Variation #3	canvas, clientRects, navigator, screen, webRTC	Consistent Variation #4
canvas, clientRects, webgl, webRTC	Consistent Variation #1	canvas, clientRects, navigator, webgl, webRTC	Consistent Variation #2
canvas, font, navigator, screen	Consistent Variation #4	canvas, clientRects, screen, webgl, webRTC	Consistent Variation #3
canvas, font, navigator, webgl	Consistent Variation #2	canvas, font, navigator, screen, webgl	Consistent Variation #4
canvas, font, navigator, webRTC	Consistent Variation #2	canvas, font, navigator, screen, webRTC	Consistent Variation #4
canvas, font, screen, webgl	Consistent Variation #3	canvas, font, navigator, webgl, webRTC	Consistent Variation #2
canvas, font, screen, webRTC	Consistent Variation #3	canvas, font, screen, webgl, webRTC	Consistent Variation #3
canvas, font, webgl, webRTC	Consistent Variation #1	canvas, navigator, screen, webgl, webRTC	Consistent Variation #4

Here, we see that while the canvas fingerprint obfuscation techniques result in a variation in the fingerprint away from the initial fingerprint calculated (when no Addons were implemented to add interference), they produce a set of consistent fingerprints even when combined with other fingerprint obfuscation techniques. These are where the fingerprint id calculated to Variation 1 is observed for the canvas obfuscation technique applied either on its own, or in conjunction with one or more of the following:

- [battery, clientRects, font, webgl, webRTC]

For the resultant fingerprint id, we receive Variation 2 if we apply canvas interference stacked

125 with navigator interference. These two interference/ obfuscation techniques can be combined
126 with the following obfuscation techniques to consistently receive the Variation 2 of a fingerprint
127 id.

128 The canvas obfuscation script enerates random mathematical noise to actively alter graphic
129 pixels so that the website visit results in a unique canvas signature whilst the navigator script
130 disables spoofing and overwrites navigator.plugins to return an empty array [] as part of
131 a blending approach to make the values generic. The consistent value implemented by the
132 navigator script is sufficient for the FingerprintJS to be considered a consistent visitor (when
133 in conjunction with the canvas obfuscation).

134 ▪ [battery, clientRects, font, webgl, webRTC]

135 For the resultant fingerprint id, we receive Variation 3 if we apply canvas interference stacked
136 with screen interference. These two interference/ obfuscation techniques can be combined with
137 the following obfuscation techniques to consistently receive the Variation 3 of a fingerprint id.

138 ▪ [battery, clientRects, font, webgl, webRTC]

139 The screen interference technique employeeed uses a crowd blending approach to force a
140 hardcoded and highly common configuration (1920x1080 resolution, 24-bit color depth). This
141 is so your browser profile looks exactly like millions of other standard desktop users. Again, we
142 see that the hardcoded values provide consistency of calculated response when a tool such as
143 FingerprintJS is responding to canvas fingerprinting obfuscation.

144 For the resultant fingerprint id, we receive Variation 4 if we apply canvas interference stacked
145 with both navigator and screen interference. These three interference/ obfuscation techniques
146 can be combined with the following obfuscation techniques to consistently receive the Variation
147 4 of a fingerprint id.

148 ▪ [battery, clientRects, font, webgl, webRTC]

149 The exception to these rules applies to canvas fingerprinting obfuscation when combined with
150 audio fingerprinting interference where the technique implemented for the audio fingerprinting
151 obfuscation interferes with the Fingerprinting software within the browser (using FingerprintJS)
152 to calculate a visitor id/ fingerprint that is unique on each browser session.

153 The difference between these obfuscation techniques is that the Audio only hooks into
154 AudioContext and OfflineAudioContext to protect against audio-based tracking. The canvas
155 obfuscation hooks into HTMLCanvasElement and CanvasRenderingContext2D to protect
156 against graphic-rendering tracking. The audio script adds fine-grain floating-point noise (-
157 0.0001 to 0.0001) and byte noise (-1 to 1) to audio frequencies and waveforms whilst the
158 canvas script manipulates the img.data[i] of rendered canvas pixels. This demonstrates that the
159 FingerprintJS fingerprinting techniques are more resilient to canvas fingerprinting interference
160 but less so to audio interference, although their fingerprint calculations can be heavily affected
161 by noise added to audi frequencies.

162 With regards to the following obfuscation techniques (presented below) that did not affect the
163 outcome of the canvas fingerprinting obfuscation technique either on its own or combined/
164 stacked with navigator or screen fingerprint obfuscation techniques.

165 ▪ [battery, clientRects, font, webgl, webRTC]

166 Each of these techniques produce a reliable randomness for the fingerprint id (as calculated
167 by the FingerprintJS opensource software), where repeated runs of each produce a different
168 fingerprint id per browser session.

169 It is worth noting that navigator and screen obfuscation techniques, when not combined with
170 our canvas obfuscation technique, also produce reliably distinct fingerprint ids for each browser
171 session.

172 AI usage disclosure

173 No generative AI tools were used in the development of this software, the writing of this
174 manuscript, or the preparation of supporting materials.

175 Acknowledgements

176 I wish to express my gratitude to Infoserv Systems for their ongoing encouragement.
177 I would also like to acknowledge the previous work in this domain by Fingerprint Defender
178 (Fracture, 2026), which I used as a basis for much of my own.

179 Eckersley, P. (2010). *How Unique Is Your Web Browser?* [https://coveryourtracks.eff.org/
180 static/browser-uniqueness.pdf](https://coveryourtracks.eff.org/static/browser-uniqueness.pdf)

181 Fingerprint. (2026a). Fingerprintjs. In *GitHub repository*. GitHub. [https://github.com/
182 fingerprintjs/fingerprintjs](https://github.com/fingerprintjs/fingerprintjs)

183 Fingerprint. (2026b). *Identify every visitor*. Fingerprint. [https://fingerprint.com/try/identify-
184 now](https://fingerprint.com/try/identify-now)

185 Fracture, D. (2026). Fingerprint defender. In *Mozilla Addons*. Firefox Browser Addons.
186 <https://addons.mozilla.org/en-GB/firefox/addon/fingerprint-defender>

187 Ganz, T. (2024). *Privacy-Preserving Canvas Fingerprinting*. [https://thecout.com/blog/canvas/
188](https://thecout.com/blog/canvas/)

189 Hibbert, C. (2022). *Your online browser fingerprint can be tracked*. [https://syn-star.co.uk/
190 cyber-security-fingerprint-browser-tracking](https://syn-star.co.uk/cyber-security-fingerprint-browser-tracking)

191 ilGur. (2026a). Font fingerprint defender. In *Mozilla Addons*. Firefox Browser Addons.
192 <https://addons.mozilla.org/en-GB/firefox/addon/font-fingerprint-defender>

193 ilGur. (2026b). WebGL fingerprint defender. In *Mozilla Addons*. Firefox Browser Addons.
194 <https://addons.mozilla.org/en-GB/firefox/addon/webgl-fingerprint-defender>

195 kkapsner. (2026). CanvasBlocker. In *Mozilla Addons*. Firefox Browser Addons. [https:
196 //addons.mozilla.org/en-GB/firefox/addon/canvasblocker](https://addons.mozilla.org/en-GB/firefox/addon/canvasblocker)

197 Omisola, I. (2025). *Canvas Fingerprinting: What Is It and How to Bypass It*. [https://www.
198 zenrows.com/blog/canvas-fingerprinting#what-is](https://www.zenrows.com/blog/canvas-fingerprinting#what-is)

199 Sayrus. (2020). AudioContext scrambler. In *Mozilla Addons*. Firefox Browser Addons.
200 <https://addons.mozilla.org/en-US/firefox/addon/audiocontext-scrambler>

201 Your web browsing habits may be less private than you think. (2023). In *IBM Research*. IBM.
<https://research.ibm.com/blog/browser-fingerprinting>